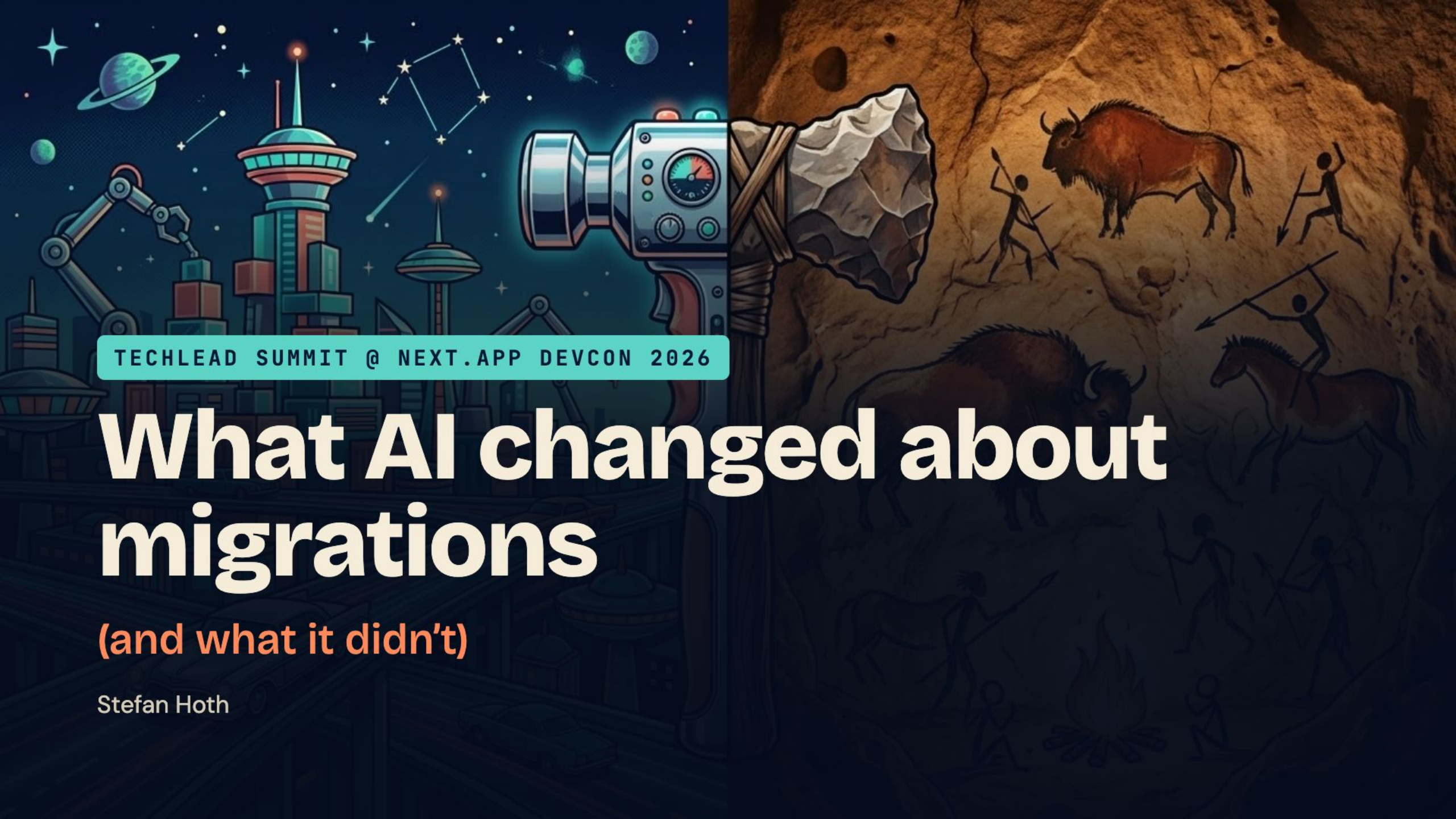




TECHLEAD SUMMIT @ NEXT.APP DEVCON 2026

# What AI changed about migrations

(and what it didn't)

Stefan Hoth

WHO'S TALKING

# Stefan Hoth

*Sooo much experience!*

---

**5+** years as contractor

---

**5+** years in Mobile

---

**5+** years in Platform Leadership

---

Add it up: 15+ years of saying "it depends."



*Me, looking up at the scope of a migration.*

THE MONOLITH

# Sound familiar?

YJTMV – Your Jenga Tower may vary.



We call this a

**Big Hairy  
Problem.**



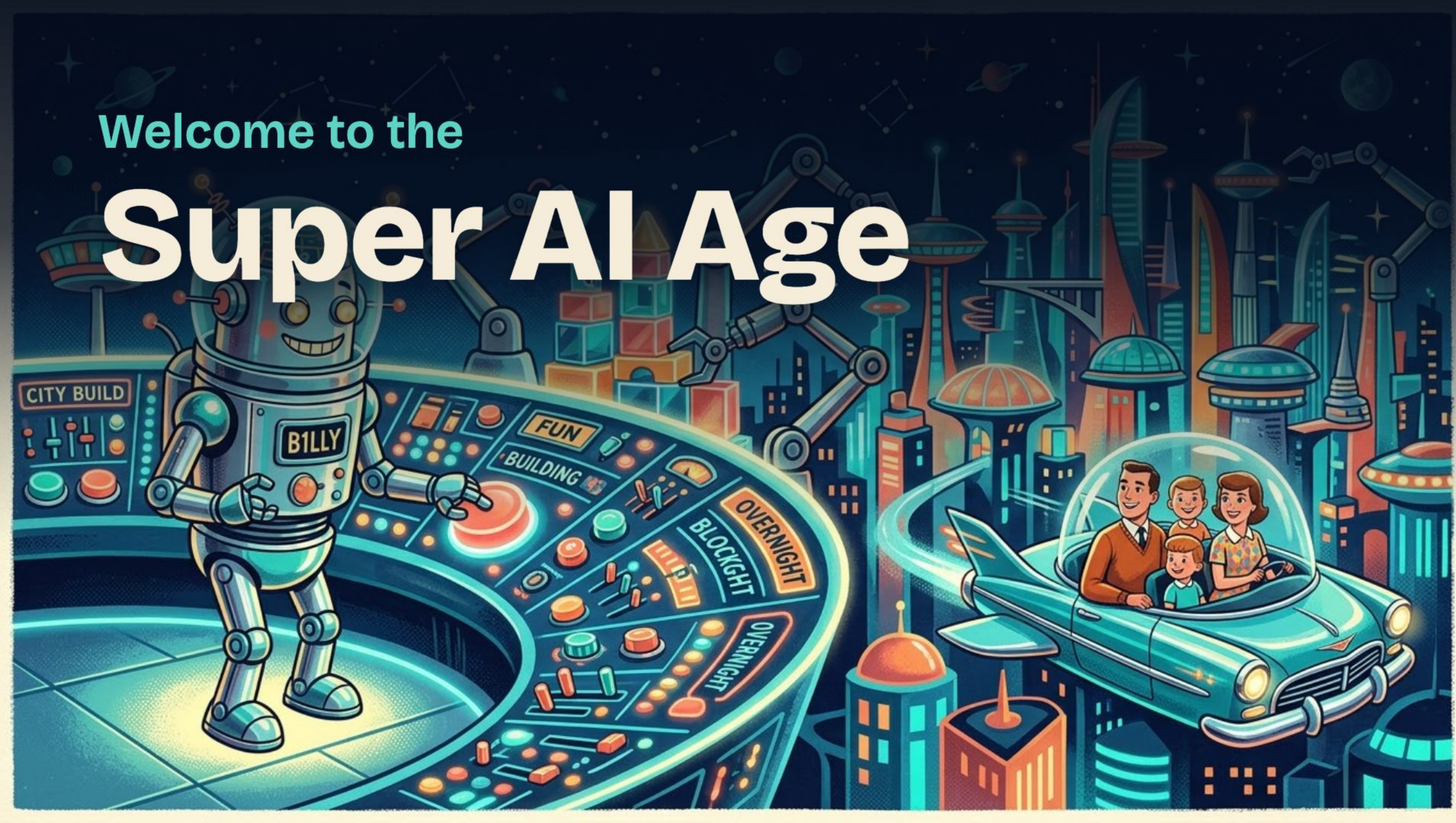
THE CLASSIC ANSWER

# So we're going to migrate.



Welcome to the

# Super AI Age



---

THE AGENT LOOP • 01 / 04

# AI turns the error into a ticket.

Summary, stack trace, repro steps.



---

THE AGENT LOOP • 02 / 04

# A coding agent builds the fix.

Branch, code, tests.



---

THE AGENT LOOP • 03 / 04

# Review agents check it.

Security, style, edge cases.



---

THE AGENT LOOP • 04 / 04

**Shipped before  
lunch. ✓**

Most of us do this every day.





# So why not:

> PROMPT

**“Migrate all our frontends to the new stack.”**

RUN ▶

*If you can prompt it, you can have it. Right?*



**BUT WAIT!**

**Meanwhile, my  
lead engineer  
is writing a  
long RFC.**

Building demo repos. Giving  
talks. Running discussions.  
**By hand.**



AI makes  
migrations **cheap.**



It doesn't make  
them **easy.**

ONCE UPON A TIME • 2022

# React inside a 15-year-old PHP monolith.

PHP owned routing, sessions, rendering.



0

local dev servers

1

button color change =  
full-stack redeploy

2x

failed Webpack 5 upgrade  
attempts

# Untangling this today?

AI would take a lot of it off my plate.

*"Oh, how I would love to go back in time and try!"*



## THE REAL PROBLEM

# Too big and too fuzzy to ever win a quarter

ROADMAP

Q1

Q2

Q3

Q4

Q1

Q2

2022

**The migration: huge, fuzzy, risky ... and then?**

Lost to smaller, well-defined projects. Every single quarter.

WITH AI

**Build it**

The implementation fits into one quarter.

**AI shrinks the investment. Now the migration can compete.**

THE PROTOTYPE



# Build a demo on live production data

2022

**weeks of work**

TODAY

**an afternoon**

# Now sell it.

Explain it. Frame it. Find a sponsor.



# Our own RFC in 2022

## PROBLEMS

slow, complex build  
slow dev server  
in-house GraphQL client  
not ready for React 18  
no SSR with client routing  
hacky async components  
two platform versions  
SSR can't run on the edge

**“Many of these issues will be knocked out at once.”**

**An open invitation to bring your own problem.**



# The Research Club

Smart people. Weekly meetings. Everyone solving a different problem.  
Nothing shipped.

# Same architecture. Same model.

## One extra sentence. Different result.

### PROMPT A

Describe the architecture

---

#1 problem

**Broken feedback loop (DX)**

Lead option

**Vite, PHP stays the router**

### PROMPT B

... + "Our biggest concern is performance and SEO."

---

#1 problem

**Performance & Core Web Vitals**

Lead option

**Strangler fig with Next.js**

## Who agreed which problem matters?

WRITE THEM DOWN

# Agreed problems

## We solve

✓ Problem 1

✓ Problem 2

✓ Problem 3

## We don't

✗ Problem 4

✗ Problem 5

✗ Problem 6

✗ Problem 7

✗ Problem 8

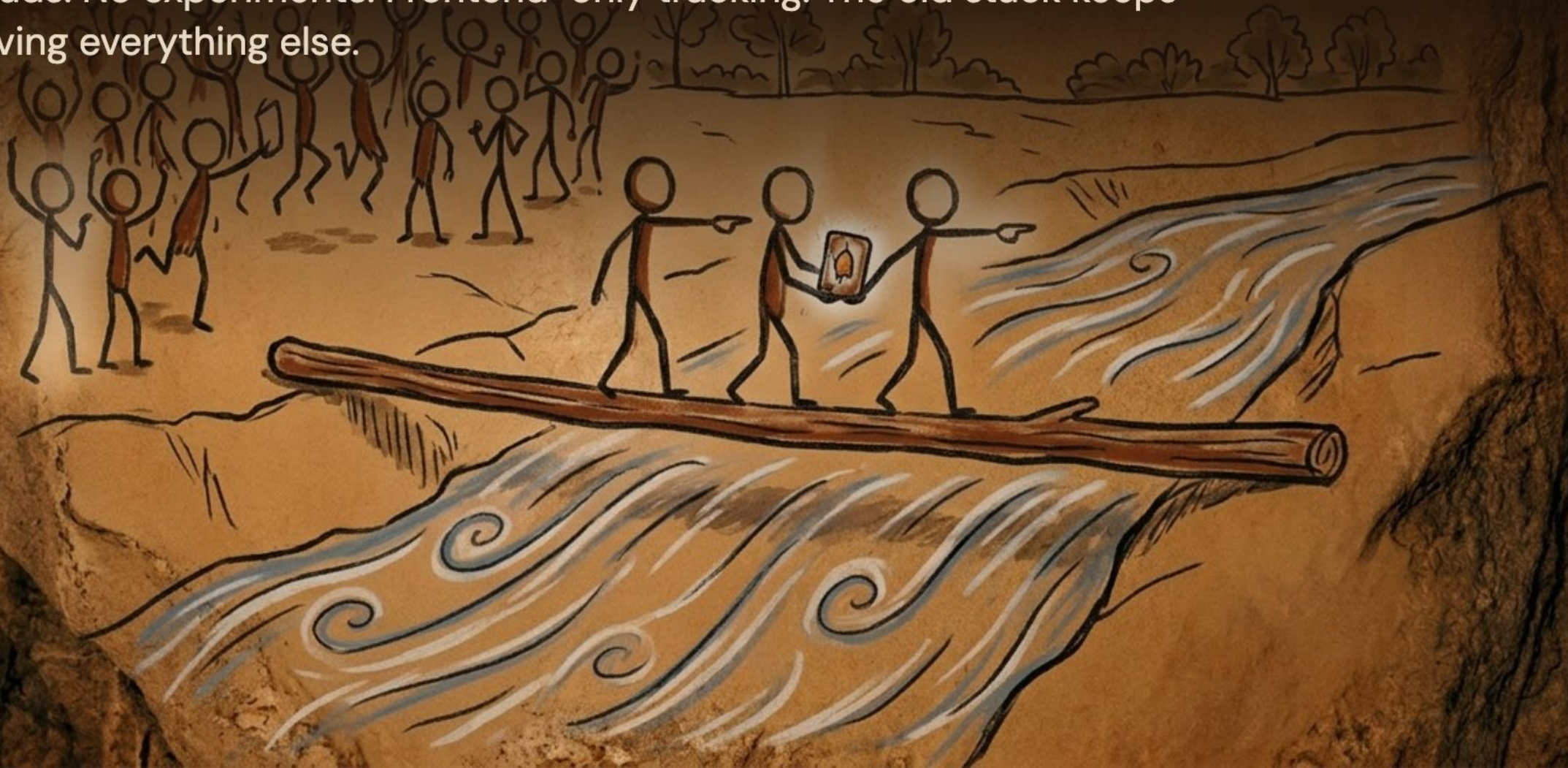
*Out of scope: for v1, for now.*

# Why the agent loop works for bugs

| WHAT IT HAS        | Bug fix | Migration |
|--------------------|---------|-----------|
| Agreed problem     | ✓ YES   | ⚠ NO      |
| One owner          | ✓ YES   | ⚠ NO      |
| Small scope        | ✓ YES   | ⚠ NO      |
| Easy rollback      | ✓ YES   | ⚠ NO      |
| Nobody to convince | ✓ YES   | ⚠ NO      |

# One tiny feature. **Four weeks.**

No ads. No experiments. Frontend-only tracking. The old stack keeps serving everything else.



# The docs were quick to write.

Who reads them? Who trains the teams? Who makes them want it?



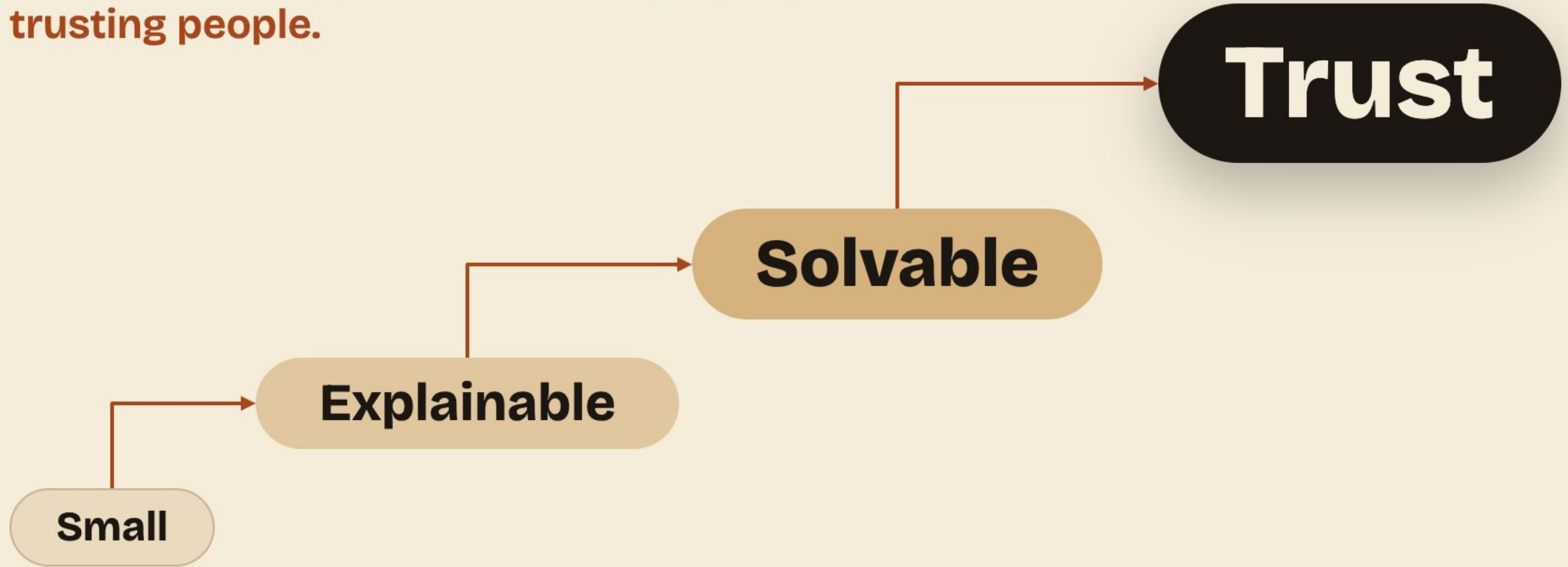
**“**

**I don't want to go back to our  
old stack anymore.**

**— PRODUCT TEAM RETRO, UNPROMPTED**

# Small makes trust possible

A migration only succeeds through people trusting people.



THOUGHT EXPERIMENT

**With today's tools, we could have rewritten the whole frontend in a sprint.**



THOUGHT EXPERIMENT

...and shipped it to a company that didn't want it.

## The Bulldozer



# Two ways AI can make it worse



## The Research Club 2.0

Prototypes without integration



## The Bulldozer

Delivery without trust or preparation

# Risk doesn't disappear. **It moves.**



CAN BE ROLLED BACK

## Code



CAN'T BE ROLLED BACK

**Trust**

**Learned skills**

**Changed processes**

**Customer experience**

BACK TO TODAY

# My lead engineer is right.

RFCs, demos, talks: that's how we build trust and understanding.



MEANWHILE,

# Your job as Engineering Leader

**1**

Get agreement on the problems first

**2**

Set the priority and create the forum

**3**

Plan next steps with leadership and peers

**4**

Organize the work so the prototype becomes production

**5**

Keep everyone informed and excited, all the way through

THE ONE THING TO TAKE HOME

**Before you touch a single line of code, do your homework.**

Which pain are you solving?

Who needs to trust you?

**Convincing the tribe hasn't changed in  
10,000 years.**

**AI won't change it either.**



# Thank you!

Feedback? Questions? **Let's connect.**

[stefanhoth.com](https://stefanhoth.com)

---

MY ARTIFICIAL CO-AUTHORS



Claude



Gemini



AssemblyAI

*They made this talk cheap. Convincing you was still my job.*



SCAN • CONNECT